

Rahul_Patil_KH

CDAC_DAC_Mumbai

ASSIGNMENT-02

Part A

What will the following commands do?

1. echo "Hello, World!"

Ans: This command prints Hello, World!"

2. name="Productive"

Ans: In name variable 'Productive' value is stored

3. touch file.txt

Ans: Create a file.txt file

4. ls -a

Ans: List all the files and directory

5. rm file.txt

Ans: remove file.txt

6. cp file1.txt file2.txt

Ans: copy content of file1.txt to file2.txt

7. mv file.txt /path/to/directory/

Ans: move the file.txt to this directory

8. chmod 755 script.sh

Ans: giving access rwx to owner , rx to group and other

9. grep "pattern" file.txt

Ans: display line containing pattern word in file.txt

10. kill PID

Ans: This would attempt to terminate the process with the PID

11. `mkdir mydir && cd mydir && touch file.txt && echo "Hello, World!" > file.txt && cat file.txt`

Ans: make a mydir directory and then go mydir also create a file.txt file and save echo

"Hello, World!" in file.txt and display the data of file.txt

12. `ls -l | grep ".txt"`

Ans:

13. `cat file1.txt file2.txt | sort | uniq`

Ans: Display Merges file1.txt and file2.txt, sorts the lines, and removes any duplicate lines, displaying the unique sorted content.

14. `ls -l | grep "^d"`

Ans: Listsing all .txt files in the current directory, showing detailed information about them

15. `grep -r "pattern" /path/to/directory/`

Ans: Display line containing pattern in this firectory

16. `cat file1.txt file2.txt | sort | uniq -d`

Ans: merge file.txt and file.txt sort it and only display uniq values

17. `chmod 644 file.txt`

Ans: giving rw access to owner ,r access to group and r access to other user

18. `cp -r source_directory destination_directory`

Ans: Copy the directories recursively from source directory to destination directory

19. `find /path/to/search -name "*.txt"`

Ans: Find files containing .txt

20. `chmod u+x file.txt`

Ans: changing owner access to execute

21. echo \$PATH

Ans: print value stored in path

Part B

Identify True or False:

1. ls is used to list files and directories in a directory.

Ans: True

2. mv is used to move files and directories.

Ans:

3. cd is used to copy files and directories.

Ans: False

4. pwd stands for "print working directory" and displays the current directory.

Ans: False

5. grep is used to search for patterns in files.

Ans: True

6. chmod 755 file.txt gives read, write, and execute permissions to the owner, and read and execute permissions to group and others.

Ans: True

7. mkdir -p directory1/directory2 creates nested directories, creating directory2 inside directory1 if directory1 does not exist.

Ans: True

8. rm -rf file.txt deletes a file forcefully without confirmation.

Ans: False

Identify the Incorrect Commands:

1. chmodx is used to change file permissions.

Ans: chmod is command x is added extra

2. cpy is used to copy files and directories.

Ans: cp is the command

3. mkfile is used to create a new file.

Ans: touch is used to create new file

4. catx is used to concatenate files.

Ans: cat is used to concatenate files

5. rn is used to rename files.

Ans: mv is used to rename

Part C

Question 1: Write a shell script that prints "Hello, World!" to the terminal.

Input: `echo $Hello, World!`

Output:

```
rahulpatil@rahulpatil:/media/rahulpatil/10AF-1E07/COS$ bash hello.sh
Hello, World!
rahulpatil@rahulpatil:/media/rahulpatil/10AF-1E07/COS$
```

Question 2: Declare a variable named "name" and assign the value "CDAC Mumbai" to it. Print the value of the variable.

Input: `echo "Enter a name:"`

`read name`

`echo "You entered: $name"`

Output:

```
rahulpatil@RahulPatil:~$ bash Q2.sh
Enter a name:
CDAC Mumbai
You entered: CDAC Mumbai
```

Question 3: Write a shell script that takes a number as input from the user and prints it.

Input:

`echo "Enter the number"`

`read n`

`echo "You entered number" n`

Output:

```
rahulpatil@RahulPatil:~$ bash Q3.sh
Enter the number
234
You entered number 234
```

Question 4: Write a shell script that performs addition of two numbers (e.g., 5 and 3) and prints the result.

Input:

a=5

b=3

sum=\$((a + b))

echo "5+3=" "\$sum"

Output:

```
rahulpatil@RahulPatil:~$ touch Q4.sh
rahulpatil@RahulPatil:~$ nano Q4.sh
rahulpatil@RahulPatil:~$ bash Q4.sh
5+3= 8
```

Question 5: Write a shell script that takes a number as input and prints "Even" if it is even, otherwise prints "Odd".

Input:

echo "Enter a number"

read num

if ((num%2 == 0))

then

echo "\$num is even number"

else

echo "\$num is odd number"

fi

Output:

```
rahulpatil@RahulPatil:~$ touch Q5.sh
rahulpatil@RahulPatil:~$ nano Q5.sh
rahulpatil@RahulPatil:~$ bash Q5.sh
Enter a number
23
23 is odd number
```

Question 6: Write a shell script that uses a for loop to print numbers from 1 to 5.

Input:

```
for((a=0;a<6; a++))
```

```
do
```

```
echo $a
```

```
done
```

Output:

```
rahulpatil@RahulPatil:~$ touch Q6.sh
rahulpatil@RahulPatil:~$ nano Q6.sh
rahulpatil@RahulPatil:~$ bash Q6.sh
0
1
2
3
4
5
```

Question 7: Write a shell script that uses a while loop to print numbers from 1 to 5.

Input:

```
a=1
```

```
while [ $a -le 5 ]
```

```
do
```

```
echo $a
```

```
a=$((a + 1))
```

```
done
```

Output:

```
cdac@LAPTOP-9K9CUDH3:~/Assignment2$ nano Q7.sh
cdac@LAPTOP-9K9CUDH3:~/Assignment2$ bash Q7.sh
1
2
3
4
5
cdac@LAPTOP-9K9CUDH3:~/Assignment2$ |
```

Question 8: Write a shell script that checks if a file named "file.txt" exists in the current directory. If it does, print "File exists", otherwise, print "File does not exist".

Input:

```
if [ -e "file.txt" ]; then
```

```
    echo "File exists"
```

```
else
```

```
    echo "File does not exist"
```

```
fi
```

Output:

```
rahulpatil@RahulPatil:~$ touch Q8.sh
rahulpatil@RahulPatil:~$ nano Q8.sh
rahulpatil@RahulPatil:~$ bash Q8.sh
File does not exist
```

Question 9: Write a shell script that uses the if statement to check if a number is greater than 10 and prints a message accordingly.

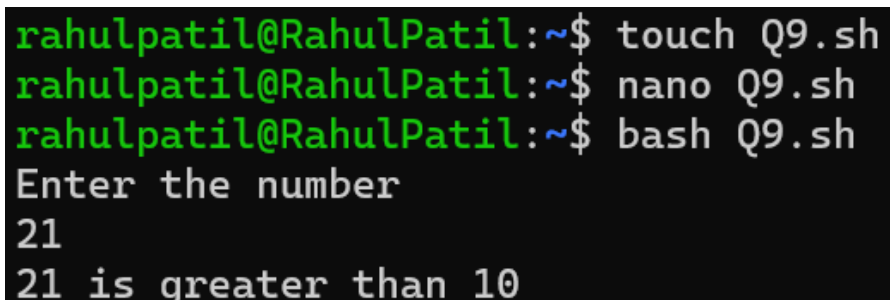
Input:

```
echo "Enter the number"
```



```
read num
if(($num > 10))
then
echo $num "is greater than 10"
elif(($num == 10))
then
echo $num "=10"
else
echo $num "is less than 10"
fi
```

Output:

A terminal window with a black background and green text. The prompt is 'rahulpatil@RahulPatil:~\$'. The user enters 'touch Q9.sh', 'nano Q9.sh', and 'bash Q9.sh'. The script prompts 'Enter the number' and the user enters '21'. The script outputs '21 is greater than 10'.

```
rahulpatil@RahulPatil:~$ touch Q9.sh
rahulpatil@RahulPatil:~$ nano Q9.sh
rahulpatil@RahulPatil:~$ bash Q9.sh
Enter the number
21
21 is greater than 10
```

Question 10: Write a shell script that uses nested for loops to print a multiplication table for numbers from 1 to 5. The output should be formatted nicely, with each row representing a number and each column representing the multiplication result for that number.

Input:

```
for i in {1..5}
do
  for j in {1..10}
  do
    echo $i "*" $j "=" $((i * j))
```

done

echo

done

Output:

```
rahulpatil@RahulPatil:~$ touch Q10.sh
rahulpatil@RahulPatil:~$ nano Q10.sh
rahulpatil@RahulPatil:~$ bash Q10.sh
1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
1 * 4 = 4
1 * 5 = 5
1 * 6 = 6
1 * 7 = 7
1 * 8 = 8
1 * 9 = 9
1 * 10 = 10

2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
2 * 10 = 20

3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
```

Question 11: Write a shell script that uses a while loop to read numbers from the user until the user enters a negative number. For each positive number entered, print its square. Use the break statement to exit the loop when a negative number is entered.

Input:

num=0

while ((num >= 0))

do

 echo "Enter the number"

 read num

 if ((num >= 0))

 then

 s=\$((num * num))

 echo "The square of \$num is \$s"

 else

 break

 fi

done

Output:

```
rahulpatil@RahulPatil:~$ touch Q11.sh
rahulpatil@RahulPatil:~$ nano Q11.sh
rahulpatil@RahulPatil:~$ bash Q11.sh
Enter the number
123
The square of 123 is 15129
Enter the number
24
The square of 24 is 576
Enter the number
23
The square of 23 is 529
Enter the number
12
The square of 12 is 144
Enter the number
2
The square of 2 is 4
Enter the number
3
The square of 3 is 9
Enter the number
4
The square of 4 is 16
Enter the number
5
```

Part D

Common Interview Questions (Must know)

1. What is an operating system, and what are its primary functions?

Ans: An operating system (OS) is software that manages computer hardware and software resources and provides common services for computer programs. Its primary functions include managing hardware resources, providing an interface for user interaction, enabling multitasking, ensuring security and access control, and managing files and storage.

2. Explain the difference between process and thread.

Ans: A process is an independent program in execution with its own memory space, while a thread is a smaller unit of a process that shares the same memory space. Threads within a process can communicate with each other more easily than processes.

3. What is virtual memory, and how does it work?

Ans: Virtual memory is a memory management technique that creates an "illusion" for users that they have more RAM than physically available. It uses disk space to extend physical memory, swapping data between RAM and the disk as needed.

4. Describe the difference between multiprogramming, multitasking, and multiprocessing.

Ans:

- Multiprogramming: Running multiple programs simultaneously by managing the CPU time for each.
- Multitasking: A more refined version of multiprogramming, allowing more efficient process management by switching between tasks.
- Multiprocessing: The use of multiple processors to perform multiple tasks simultaneously, enhancing performance.

5. What is a file system, and what are its components?

Ans: A file system is a method for storing and organizing files on a storage device. Its components include files, directories, metadata, and access control information.

6. What is a deadlock, and how can it be prevented?

Ans: A deadlock is a situation where two or more processes are blocked, each waiting for the other to release resources. Prevention strategies include avoiding circular wait, requiring resources to be requested in a specific order, and using timeouts.

7. Explain the difference between a kernel and a shell.

Ans: The kernel is the core part of the OS that manages hardware and system resources. The shell is a user interface that allows users to interact with the OS, typically through commands.

8. What is CPU scheduling, and why is it important?

Ans: CPU scheduling is the process of determining which process will use the CPU at a given time. It is important to optimize the use of CPU time and ensure fairness and efficiency in process execution.

9. How does a system call work?

Ans: A system call is a mechanism that allows user programs to request services from the operating system, such as file manipulation or process management. It typically involves a context switch to kernel mode.

10. What is the purpose of device drivers in an operating system?

Ans: Device drivers allow the operating system to communicate with hardware devices, translating OS-level commands into hardware-specific instructions.

11. Explain the role of the page table in virtual memory management.

Ans: A page table is used to map virtual memory addresses to physical memory addresses. It helps the operating system manage virtual memory by tracking which pages are loaded in physical memory.

12. What is thrashing, and how can it be avoided?

Ans: Thrashing occurs when the operating system spends too much time swapping data between RAM and disk, causing performance degradation. It can be avoided by using algorithms to manage memory allocation and reducing the number of processes in the system.

13. Describe the concept of a semaphore and its use in synchronization.

Ans: A semaphore is a synchronization primitive used to control access to a shared resource by multiple processes in a concurrent system. It helps avoid race conditions by using signaling mechanisms like wait and signal.

14. How does an operating system handle process synchronization?

Ans: Operating systems use synchronization mechanisms like semaphores, mutexes, and condition variables to ensure that processes and threads access shared resources in an orderly and predictable manner, preventing data inconsistency.

15. What is the purpose of an interrupt in operating systems?

Ans: An interrupt is a mechanism that allows the OS to respond to events like hardware requests or errors. It temporarily halts the current process to give attention to the interrupting event, improving responsiveness.

16. Explain the concept of a file descriptor.

Ans: A file descriptor is an integer used by an operating system to represent an open file or other I/O resources. It serves as a reference to access the file during read, write, and other operations.

17. How does a system recover from a system crash?

Ans: The system uses recovery techniques like transaction logs, checkpoints, and journaling to restore data consistency. Upon reboot, the system uses these logs to determine which operations were successful and which need to be redone.

18. Describe the difference between a monolithic kernel and a microkernel.

Ans: A monolithic kernel contains all the essential services and device drivers in a single large block, offering high performance but less flexibility. A microkernel has a minimal core, with other services running in user space, offering better modularity and fault isolation but possibly less performance.

19. What is the difference between internal and external fragmentation?

Ans:

- Internal fragmentation occurs when memory is allocated to a process but not fully used, leaving gaps within allocated blocks.
- External fragmentation happens when there is enough total free memory but not in contiguous blocks, making it impossible to allocate large processes.

20. How does an operating system manage I/O operations?

Ans: The OS manages I/O operations using device drivers, which control the communication between the operating system and hardware devices. It uses buffering, caching, and spooling to optimize the performance of I/O devices.

21. Explain the difference between preemptive and non-preemptive scheduling.

Ans:

- Preemptive scheduling allows the OS to interrupt a process and allocate the CPU to another process.
- Non-preemptive scheduling means once a process is assigned the CPU, it runs until completion or voluntarily relinquishes control.

22. What is round-robin scheduling, and how does it work?

Ans: Round-robin scheduling assigns a fixed time slice to each process in a cyclic order. Once the time slice expires, the next process in the queue is given a turn to execute.

23. Describe the priority scheduling algorithm. How is priority assigned to processes?

Ans: Priority scheduling assigns each process a priority. The CPU is allocated to the process with the highest priority. Priorities can be static or dynamic, and can be based on factors like process importance or resource requirements.

24. What is the shortest job next (SJN) scheduling algorithm, and when is it used?

Ans: The SJN algorithm selects the process with the shortest burst time (execution time) next. It minimizes average waiting time but requires knowing the exact burst time, making it difficult to implement in practice.

25. Explain the concept of multilevel queue scheduling.

Ans: Multilevel queue scheduling divides processes into different queues based on priorities or types. Each queue has its own scheduling algorithm, and processes are scheduled from the highest-priority queue to the lowest.

26. What is a process control block (PCB), and what information does it contain?

Ans: A PCB is a data structure that stores information about a process, such as its process ID, state, program counter, registers, memory management information, and scheduling information.

27. Describe the process state diagram and the transitions between different process states.

Ans: A process can be in one of several states, including New, Ready, Running, Waiting, and Terminated. Transitions occur based on events like process creation, CPU allocation, I/O requests, and process completion.

28. How does a process communicate with another process in an operating system?

Ans: Processes can communicate through inter-process communication (IPC) mechanisms like pipes, message queues, shared memory, and sockets.

29. What is process synchronization, and why is it important?

Ans: Process synchronization ensures that multiple processes or threads can operate concurrently without causing conflicts or inconsistent results when accessing shared resources.

30. Explain the concept of a zombie process and how it is created.

Ans: A zombie process is a terminated process that still has an entry in the process table because its parent has not read its exit status. It is created when a parent process fails to call `wait ( )` for its terminated child process.

31. Describe the difference between internal fragmentation and external fragmentation.

Ans: See Q19 for details.

32. What is demand paging, and how does it improve memory management efficiency?

Ans: Demand paging loads pages into memory only when they are needed, reducing memory usage and improving efficiency by not loading unnecessary pages.

33. Explain the role of the page table in virtual memory management.

Ans: The page table maps virtual addresses to physical addresses, helping the OS manage virtual memory and enabling efficient access to data stored in non-contiguous memory locations.

34. How does a memory management unit (MMU) work?

Ans: The MMU translates virtual memory addresses into physical memory addresses, enabling the OS to manage virtual memory efficiently.

35. What is thrashing, and how can it be avoided in virtual memory systems?

Ans: Thrashing occurs when the system spends more time swapping data than executing processes. It can be avoided by managing the number of processes and ensuring sufficient physical memory is available.

36. What is a system call, and how does it facilitate communication between user programs and the operating system?

Ans: A system call provides an interface for user programs to request services from the operating system, enabling interaction between user space and kernel space.

37. Describe the difference between a monolithic kernel and a microkernel.

Ans: See Q18 for details.

38. How does an operating system handle I/O operations?

Ans: See Q20 for details.

39. Explain the concept of a race condition and how it can be prevented.

Ans: A race condition occurs when multiple processes access shared resources concurrently, leading to unpredictable results. It can be prevented using synchronization mechanisms like locks or semaphores.

40. Describe the role of device drivers in an operating system.

Ans: See Q10 for details.

41. What is a zombie process, and how does it occur? How can a zombie process be prevented?

Ans: See Q30 for details.

42. Explain the concept of an orphan process. How does an operating system handle orphan processes?

Ans: An orphan process is a process whose parent has terminated. The OS usually adopts orphan processes by assigning them a new parent (usually the init process).

43. What is the relationship between a parent process and a child process in the context of process management?

Ans: A parent process creates child processes, which inherit certain properties from the parent. The parent can communicate with or control the child processes.

44. How does the fork() system call work in creating a new process in Unix-like operating systems?

Ans: The fork() system call creates a new process by duplicating the calling process, creating a child process with a new process ID.

45. Describe how a parent process can wait for a child process to finish execution.

Ans: The parent process can use the `wait ( )` system call to pause its execution until the child process completes.

46. What is the significance of the exit status of a child process in the `wait()` system call?

Ans: The exit status provides information about whether the child process terminated successfully or with an error. The parent process can retrieve this status using the `wait ( )` system call.

47. How can a parent process terminate a child process in Unix-like operating systems?

Ans: A parent process can terminate a child process using the `kill ( )` system call, sending a signal to the child to terminate.

48. Explain the difference between a process group and a session in Unix-like operating systems.

Ans: A process group is a collection of processes that can receive signals together, while a session is a collection of process groups, typically associated with a terminal session.

49. Describe how the `exec()` family of functions is used to replace the current process image with a new one.

Ans: The `exec ( )` family of functions loads a new program into the current process, replacing its image, memory, and execution flow with that of the new program.

50. What is the purpose of the `waitpid()` system call in process management? How does it differ from `wait()`?

Ans: `waitpid ( )` allows a parent process to wait for a specific child process to terminate, whereas `wait ( )` waits for any child process to terminate.

51. How does process termination occur in Unix-like operating systems?

Ans: Process termination occurs when a process completes its execution or is terminated by a signal. It releases resources and updates the process table.

52. What is the role of the long-term scheduler in the process scheduling hierarchy? How does it influence the degree of multiprogramming in an operating system?

Ans: The long-term scheduler determines which processes are admitted to the system, controlling the degree of multiprogramming by regulating the number of processes in the ready queue

53. How does the short-term scheduler differ from the long-term and medium-term schedulers in terms of frequency of execution and the scope of its decisions?

Ans: The short-term scheduler (or CPU scheduler) runs most frequently and decides which process in the ready queue will be executed next. The long-term scheduler controls process admission, and the medium-term scheduler deals with swapping processes in and out of memory.

54. Describe a scenario where the medium-term scheduler would be invoked and explain how it helps manage system resources more efficiently.

Ans: The medium-term scheduler may be invoked during heavy memory usage to swap out less important processes to disk to make room for higher-priority processes, improving system performance.

Part E

1. Consider the following processes with arrival times and burst times:

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	6

Calculate the average waiting time using First-Come, First-Served (FCFS) scheduling.

Q1) - Gantt chart

Process	A.T	B.T	R.T	W.T	T.A.T
P1	0	5	0	0	5
P2	1	3	5	4	7
P3	2	6	8	6	12

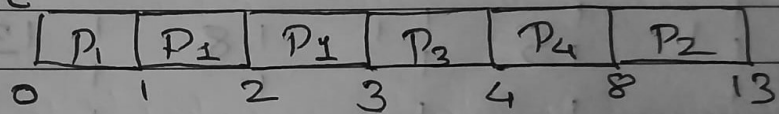
Avg W.T = $\frac{0+4+6}{3} = \frac{10}{3} = 3.33$

2. Consider the following processes with arrival times and burst times:

Process	Arrival Time	Burst Time
P1	0	3
P2	1	5
P3	2	1
P4	3	4

Calculate the average turnaround time using Shortest Job First (SJF) scheduling.

2) Gantt chart



Process	A.T	B.T	R.T	W.T	T.A.T
P1	0	3	0	0	3
P2	1	5	8	4	9
P3	2	1	3	3	4
P4	3	4	4	8	12

$$\text{Avg T.A.T} = \frac{3+9+4+12}{4} = \frac{28}{4} = 7$$

3. Consider the following processes with arrival times, burst times, and priorities (lower number indicates higher priority):

Process	Arrival Time	Burst Time	Priority
P1	0	6	3
P2	1	4	1
P3	2	7	4
P4	3	2	2

Calculate the average waiting time using Priority Scheduling.

Q9] -

Process	Priority	A.T	B.T	R.T	W.T	T.A.T
P1	3	0	6	0	6	12
P2	1	1	4	1	0	4
P3	4	2	7	12	10	17
P4	2	3	2	5	2	4

Avg W.T = $\frac{6+0+10+2}{4} = \frac{18}{4} = 4.5$

Gantt Chart

P1	P2	P4	P1	P3
0	1	5	7	12
				19

4. Consider the following processes with arrival times and burst times, and the time quantum for Round Robin scheduling is 2 units:

Process	Arrival	Burst
P1	0	4
P2	1	5
P3	2	2
P4	3	3

Calculate the average turnaround time using Round Robin scheduling.

Q4] Gantt chart

	P ₁	P ₂	P ₃	P ₁	P ₄	P ₂	P ₄	P ₂	
	0	2	4	6	8	10	12	13	14

Process	A.T	B.T	R.T	W.T	TAT
P ₁	0	4	0	4	8
P ₂	1	5	1	8	13
P ₃	2	2	4	2	4
P ₄	3	3	5	8	11

$$\text{Avg. TAT} = \frac{8 + 13 + 4 + 11}{4} = \frac{36}{4} = 9$$

$$\text{Avg. T.A.T} = \frac{8 + 13 + 4 + 11}{4} = \frac{36}{4} = 9$$

5. Consider a program that uses the fork() system call to create a child process. Initially, the parent process has a variable x with a value of 5. After forking, both the parent and child processes increment the value of x by 1. What will be the final values of x in the parent and child processes after the fork() call?

Ans:

1. Before the fork() call:

- o The parent process has a variable x with a value of 5.

2. After the fork() call:

- o The parent process and the child process both have a variable x with an initial value of 5.

3. Incrementing the value of x by 1 in both processes:

- o In the parent process, x becomes 6.
- o In the child process, x also becomes 6.

Therefore, the final value of x in both the parent and child processes after the `fork()` call and the increment operations will be 6.